

Entrega completa del proyecto

Inventario App

Pipeline CI/CD con Docker, GitHub Actions y Kubernetes

Integrantes	José Vanegas y Miguel Vanegas
Repositorio público	github.com/migu145/inventario-app
Fecha	28 de julio de 2026
Contenido	Informe técnico, evidencias, métricas DORA y comandos de demostración

Este documento reúne el trabajo realizado durante la práctica. El informe corto de reflexión se mantiene como entregable independiente; aquí se presenta el recorrido completo y un guion de comandos para demostrar el funcionamiento.

Contenido

1. Alcance del trabajo
2. Pruebas locales y aplicación
3. Docker y pipeline CI/CD
4. Seguridad, GHCR y Minikube
5. Kubernetes, persistencia y Rolling Update
6. Blue-Green y readiness
7. Métricas DORA y problemas reales
8. Comandos para la demostración

1. Alcance del trabajo

La práctica comenzó con una aplicación Node.js que administra un catálogo de productos. A partir de esa base se prepararon pruebas automáticas, una imagen Docker multi-stage y un pipeline con dos jobs. El pipeline prueba el código, construye la imagen, la analiza con Trivy y publica etiquetas inmutables por SHA en GHCR.

Después se desplegó la aplicación en Minikube. Se probó un Rolling Update con dos réplicas y una estrategia Blue-Green con dos Deployments separados. El Service decide qué versión recibe el tráfico mediante la etiqueta slot. También se agregó un Secret, un readiness real con respuesta HTTP 503 durante el arranque y una prueba de persistencia al recrear un pod.

Componente	Implementación	Comprobación
Pruebas	Node test runner, 6 casos	6 pass, 0 fail
Docker	Build multi-stage con Node 24 Alpine	La etapa test es obligatoria
CI/CD	Jobs build-test y build-push	Fail-fast, Trivy y GHCR
Kubernetes	Rolling Update, 2 réplicas	2/2 disponibles
Blue-Green	Blue v1 y Green v2	Cambio y rollback por selector
Buenas prácticas	Secret, readiness y SHA inmutable	Clave no expuesta y 503 antes de Ready

Estructura principal del repositorio

```
.github/workflows/ci-cd.yml
Dockerfile
server.js
server.test.js
k8s/deployment.yaml
k8s/service.yaml
k8s/blue-green/deployment-blue.yaml
k8s/blue-green/deployment-green.yaml
evidencias/capturas-reales/
evidencias/dora-deployments.csv
output/pdf/
```

El README contiene los comandos completos y relaciona cada demostración con su captura. Las evidencias usadas en este documento no muestran tokens ni el valor de API_KEY.

2. Pruebas locales y funcionamiento de la aplicación

La suite inicia el servidor en un puerto temporal. Comprueba /health, el arranque lento, /version, la creación y eliminación de productos y la validación de campos obligatorios. El resultado final fue de seis pruebas correctas y ninguna falla.

```
PS C:\Users\Fr4nk\Downloads\inventario-app\inventario-app> npm.cmd test

> inventario-app@1.0.0 test
> node --test

✓ GET /health responde 200 y status ok (22.0445ms)
✓ GET /health responde 503 durante el arranque lento y 200 al terminar (5.3582ms)
✓ GET /version responde con version y color (8.1014ms)
✓ POST /api/products crea un producto y GET /api/products lo lista (15.3075ms)
✓ DELETE /api/products/:id elimina el producto (7.7629ms)
✓ POST /api/products sin name/sku responde 400 (3.2545ms)
i tests 6
i suites 0
i pass 6
i fail 0
i cancelled 0
i skipped 0
i todo 0
i duration ms 288.4075
```

Evidencia: npm test termina con 6 pruebas correctas.

Endpoint	Uso	Resultado esperado
/	Interfaz del catálogo	HTML de la aplicación
/health	Salud y readiness	200 ok o 503 starting
/version	Versión, color y pod	v1/blue o v2/green
/api/products	Catálogo en JSON	Lista de productos

Resultado observado

La aplicación respondió correctamente en el puerto 3000. /version también indicó si API_KEY estaba configurada mediante un booleano, sin devolver la clave. Esta misma ruta se utilizó después para identificar qué versión atendía el tráfico en Kubernetes.

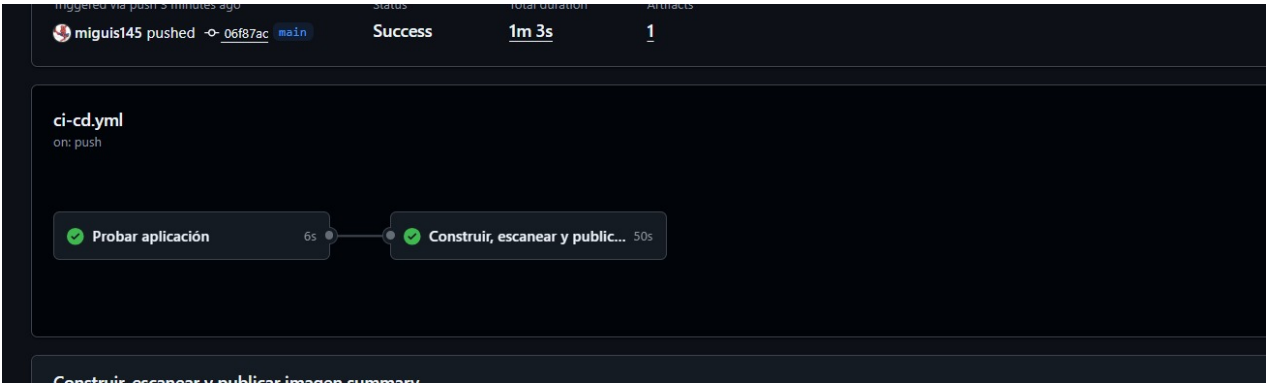
3. Docker y pipeline de GitHub Actions

El Dockerfile tiene dos etapas. La primera instala todas las dependencias y ejecuta las pruebas. La segunda instala solo producción y copia el código desde la etapa de test. Esa dependencia evita que BuildKit genere la imagen final si la suite falla.

```
PS C:\Users\Fr4nk\Downloads\Inventario-app\Inventario-app> docker build -t inventario-app:local .
[+] Building 1.7s (17/17) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 1.58kB
=> [internal] load metadata for docker.io/library/node:24-alpine3.24
=> [auth] library/node:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 348
=> [test 1/7] FROM docker.io/library/node:24-alpine3.24@sha256:a0b9bf06e4e6193cf7a0f58816cc935ff8c2a908f81e6f1a95432d679c54fbfd
=> => resolve docker.io/library/node:24-alpine3.24@sha256:a0b9bf06e4e6193cf7a0f58816cc935ff8c2a908f81e6f1a95432d679c54fbfd
=> [internal] load build context
=> => transferring context: 293B
=> CACHED [test 2/7] WORKDIR /app
=> CACHED [test 3/7] COPY package*.json ./
=> CACHED [production 4/7] RUN npm ci --omit=dev && npm cache clean --force && rm -rf /usr/local/lib/node_modules/npm && rm -f /usr/local/bin/npm
=> CACHED [test 4/7] RUN npm ci
=> CACHED [test 5/7] COPY server.js db.js server.test.js ./
=> CACHED [test 6/7] COPY public ./public
=> CACHED [test 7/7] RUN npm test && echo "Pruebas completadas"
=> CACHED [production 5/7] COPY --from=test /app/server.js /app/db.js ./
=> CACHED [production 6/7] COPY --from=test /app/public ./public
=> CACHED [production 7/7] RUN mkdir -p /app/data
=> exporting image
=> => exporting layers
=> => exporting manifest sha256:08f83456a720973323febe46c5036184e4ee1783ac8fbfc1b533dbf3b3228b12
=> => exporting config sha256:733000eb58a6e2a6dd4a86c10f9c1801615e09a71f0a1c9fdf2caa3dd797c1b6
=> => exporting attestation manifest sha256:9f263103d1d6ad0704f2631f4cb02119c2c8ae00d522a1145597b9b102cd26a
=> => exporting manifest list sha256:55b5fd92646fc0204b3e73dbc05bae2f0b06fb7ec124ce03dff55e9d5fe26e4b
=> => naming to docker.io/library/inventario-app:local
=> => unpacking to docker.io/library/inventario-app:local
View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/z0kv8lbfmsbiu37ben950mevm
```

Evidencia: construcción multi-stage y ejecución de la etapa test.

GitHub Actions repite la misma idea. El job build-test ejecuta npm ci y npm test. build-push depende de ese job, construye la imagen, ejecuta Trivy y publica en GHCR solo cuando los controles anteriores terminan correctamente.

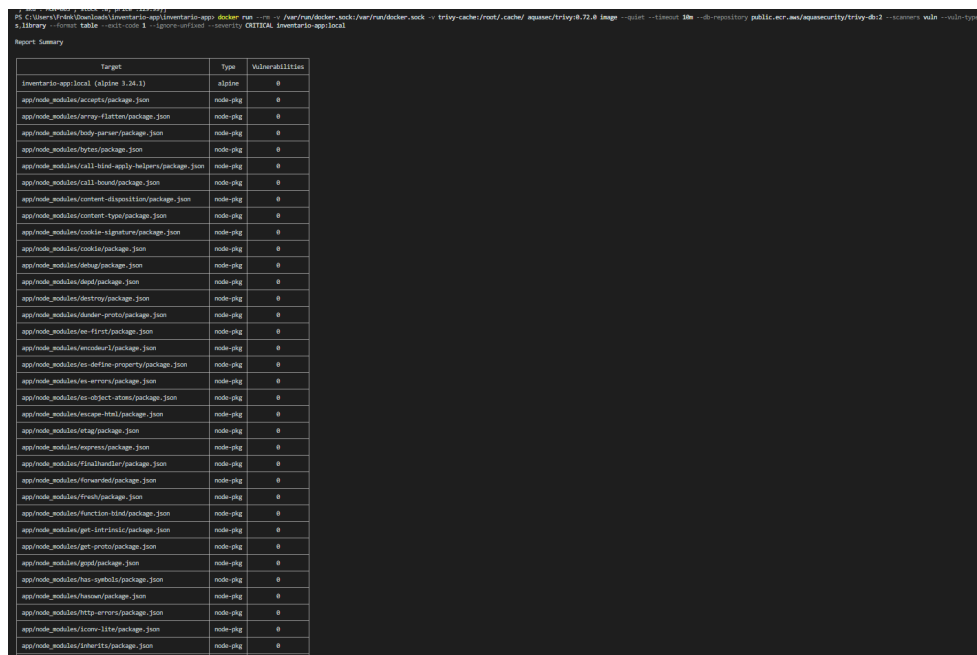


Evidencia: los dos jobs del pipeline terminan correctamente.

Job	Pasos principales	Qué detiene el flujo
build-test	Checkout, Node 24, npm ci y npm test	Cualquier prueba fallida
build-push	Buildx, Trivy, login y push	CRITICAL o error de construcción

4. Seguridad, GHCR y preparación del clúster

Trivy analiza vulnerabilidades del sistema operativo y bibliotecas. La política usada detiene el pipeline si encuentra una vulnerabilidad CRITICAL con corrección disponible. La verificación final usó la base publicada en public.ecr.aws y reportó cero hallazgos CRITICAL.



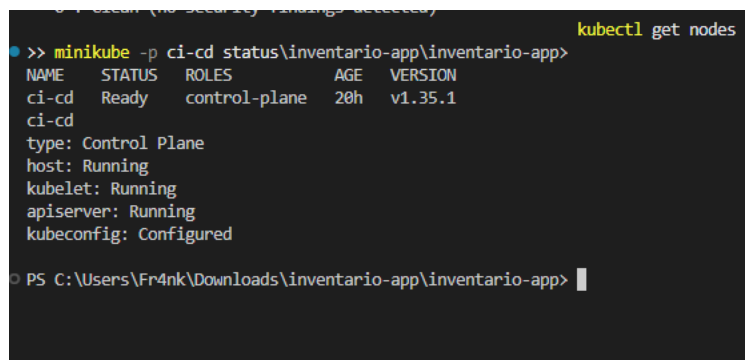
Target	Type	Vulnerabilities
inventario-app:local (alpine 3.24.1)	alpine	0
app/node_modules/accepts/package.json	node-pkg	0
app/node_modules/array-flatten/package.json	node-pkg	0
app/node_modules/body-parser/package.json	node-pkg	0
app/node_modules/buffer/package.json	node-pkg	0
app/node_modules/call-bind/package.json	node-pkg	0
app/node_modules/call-bound/package.json	node-pkg	0
app/node_modules/content-disposition/package.json	node-pkg	0
app/node_modules/content-type/package.json	node-pkg	0
app/node_modules/cookie/package.json	node-pkg	0
app/node_modules/cookie-signature/package.json	node-pkg	0
app/node_modules/cors/package.json	node-pkg	0
app/node_modules/debug/package.json	node-pkg	0
app/node_modules/depd/package.json	node-pkg	0
app/node_modules/descriptor/package.json	node-pkg	0
app/node_modules/dotenv/package.json	node-pkg	0
app/node_modules/encodeurl/package.json	node-pkg	0
app/node_modules/es-define-property/package.json	node-pkg	0
app/node_modules/es-errors/package.json	node-pkg	0
app/node_modules/es-object-atoms/package.json	node-pkg	0
app/node_modules/escape-html/package.json	node-pkg	0
app/node_modules/etag/package.json	node-pkg	0
app/node_modules/express/package.json	node-pkg	0
app/node_modules/finalhandler/package.json	node-pkg	0
app/node_modules/forwarded/package.json	node-pkg	0
app/node_modules/fresh/package.json	node-pkg	0
app/node_modules/function-bind/package.json	node-pkg	0
app/node_modules/get-intrinsic/package.json	node-pkg	0
app/node_modules/get-proto/package.json	node-pkg	0
app/node_modules/gopd/package.json	node-pkg	0
app/node_modules/has-symbol/package.json	node-pkg	0
app/node_modules/hasown/package.json	node-pkg	0
app/node_modules/http-errors/package.json	node-pkg	0
app/node_modules/iconv-lite/package.json	node-pkg	0
app/node_modules/inherits/package.json	node-pkg	0

Evidencia: resumen del escaneo Trivy sin vulnerabilidades CRITICAL.

Las imágenes se publican como ghcr.io/migu145/inventario-app:SHA y también como latest. Los manifiestos utilizan SHA porque es una referencia inmutable. latest queda disponible para descarga manual, pero no se usa para identificar un despliegue medido.

Minikube y Secret

El clúster local se ejecutó con el perfil ci-cd sobre Docker. Antes de desplegar se creó inventario-secret. Los Deployments leen API_KEY mediante secretKeyRef. La comprobación muestra solo los metadatos del Secret y el booleano apiKeyConfigured.



```

PS C:\Users\Fr4nk\Downloads\inventario-app\inventario-app> minikube -p ci-cd status
NAME      STATUS    ROLES    AGE     VERSION
ci-cd     Ready     control-plane  20h    v1.35.1
ci-cd
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
PS C:\Users\Fr4nk\Downloads\inventario-app\inventario-app>

```

Evidencia: nodo Ready y componentes de Minikube en ejecución.

5. Kubernetes, Rolling Update y persistencia

El Deployment base usa dos réplicas, maxUnavailable igual a 1 y maxSurge igual a 1. La readinessProbe evita que un pod reciba tráfico antes de estar disponible. Se promovieron dos SHA distintos y ambos rollouts terminaron con 2/2 réplicas.

```
PS C:\Users\Fr4nk\Downloads\inventario-app\inventario-app> kubectl apply -f k8s/deployment.yaml
>> kubectl rollout restart deployment/inventario-app
>> kubectl rollout status deployment/inventario-app --timeout=240s
>> kubectl get deployment inventario-app -o wide
>> kubectl get pods -l app=inventario-app --show-labels
deployment.apps/inventario-app created
deployment.apps/inventario-app restarted
Waiting for deployment "inventario-app" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "inventario-app" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "inventario-app" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "inventario-app" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "inventario-app" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "inventario-app" rollout to finish: 1 of 2 updated replicas are available...
deployment "inventario-app" successfully rolled out
NAME READY UP-TO-DATE AVAILABLE AGE CONTAINERS IMAGES SELECTOR
inventario-app 2/2 2 2 25s inventario-app ghcr.io/miguels145/inventario-app:85eaa0fc3990f21c4fd8013e44ba51e0a30ca727 app=inventario-app
NAME READY STATUS RESTARTS AGE LABELS
inventario-app-6b7c9d6886-kzvtf 1/1 Running 0 25s app=inventario-app,pod-template-hash=6b7c9d6886
inventario-app-6b7c9d6886-mwkpj 1/1 Running 0 25s app=inventario-app,pod-template-hash=6b7c9d6886
inventario-app-6f575f4468-flkps 1/1 Terminating 0 25s app=inventario-app,pod-template-hash=6f575f4468
inventario-app-blue-c4864fdb6-62fpg 1/1 Running 0 18m app=inventario-app,pod-template-hash=c4864fdb6,slot=blue
inventario-app-blue-c4864fdb6-r7cgn 1/1 Running 0 18m app=inventario-app,pod-template-hash=c4864fdb6,slot=blue
inventario-app-green-66b66dd7f4-jdngx 1/1 Running 0 18m app=inventario-app,pod-template-hash=66b66dd7f4,slot=green
inventario-app-green-66b66dd7f4-jskkk 1/1 Running 0 18m app=inventario-app,pod-template-hash=66b66dd7f4,slot=green
```

Evidencia: Rolling Update con dos imágenes SHA y 2/2 réplicas.

Acceso y respuesta

El Service expone el puerto 80 dentro del clúster. Para la demostración local se usa kubectl port-forward hacia 127.0.0.1:8080. Desde otra terminal se consultan /health, /version y /api/products.

Qué ocurrió al recrear un pod

Se agregó un producto y luego se eliminó el pod que había guardado el archivo. Kubernetes creó un reemplazo, pero el producto desapareció. db.js escribe el catálogo dentro del contenedor y no existe un PersistentVolume. Además, cada réplica conserva su propia copia. El resultado demuestra que el Deployment recupera el proceso, no los datos locales.

Antes de eliminar	Después de eliminar	Conclusión
Producto visible en el pod	Pod nuevo con datos iniciales	Se necesita almacenamiento compartido

6. Estrategia Blue-Green

Blue ejecuta v1 y Green ejecuta v2. Ambos Deployments permanecen activos. El Service incluye app=inventario-app y un selector slot. Cambiar slot de blue a green promueve la nueva versión sin reconstruir imágenes. Volver a blue realiza el rollback.

```
PS C:\Users\Fr4nk\Downloads\inventario-app\inventario-app> kubectl get deployments
>> kubectl get pods -l app=inventario-app --show-labels
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
inventario-app       2/2     2             2           7m32s
inventario-app-blue  2/2     2             2           26m
inventario-app-green 2/2     2             2           26m
NAME                READY   STATUS    RESTARTS   AGE   LABELS
inventario-app-6b7c9d6886-kzvtf  1/1    Running   0           7m32s  app=inventario-app,pod-template-hash=6b7c9d6886
inventario-app-6b7c9d6886-mwkpk  1/1    Running   0           7m32s  app=inventario-app,pod-template-hash=6b7c9d6886
inventario-app-blue-c4864fdb6-62fqg 1/1    Running   0           26m    app=inventario-app,pod-template-hash=c4864fdb6,slot=blue
inventario-app-blue-c4864fdb6-r7cgn 1/1    Running   0           26m    app=inventario-app,pod-template-hash=c4864fdb6,slot=blue
inventario-app-green-66b66dd7f4-jdrgrx 1/1    Running   0           26m    app=inventario-app,pod-template-hash=66b66dd7f4,slot=green
inventario-app-green-66b66dd7f4-jskkk 1/1    Running   0           26m    app=inventario-app,pod-template-hash=66b66dd7f4,slot=green
```

Evidencia: Blue y Green activos con dos pods por versión.

```
PS C:\Users\Fr4nk\Downloads\inventario-app\inventario-app> kubectl get secret inventario-secret
>> kubectl get service inventario-service -o jsonpath="{.spec.selector.slot}"
>> Write-Host
>> kubectl exec deployment/inventario-app-green -- node -e "fetch('http://inventario-service/version').then(r=>r.text()).then(console.log)"
NAME                TYPE      DATA   AGE
inventario-secret   Opaque    1       22h
green
{"version":"v2","color":"green","hostname":"inventario-app-green-66b66dd7f4-jdrgrx","apiKeyConfigured":true}
```

Evidencia: selector Green, respuesta v2 y API_KEY configurada.

Estado	Selector del Service	Respuesta de /version
Inicial	slot=blue	v1, blue
Promoción	slot=green	v2, green
Rollback	slot=blue	v1, blue
Estado final	slot=green	v2, green

Se eligió Blue-Green porque /version hace visible qué ambiente respondió y porque el rollback se puede demostrar cambiando un solo selector. Canary habría requerido repartir tráfico y tomar varias muestras para justificar el porcentaje recibido por cada versión.

7. Readiness y problemas encontrados

STARTUP_DELAY_SECONDS simula un arranque lento. Durante ese tiempo /health devuelve HTTP 503 y status starting. Cuando termina, devuelve 200 y status ok. Kubernetes mantiene el pod en Running 0/1 hasta que la sonda recibe la respuesta correcta.

```
PS C:\Users\Fr4nk\Downloads\inventario-app\inventario-app> kubectl rollout restart deployment/inventario-app-green
>> kubectl get pods -l app=inventario-app,slot=green -w
deployment.apps/inventario-app-green restarted
NAME                                READY   STATUS             RESTARTS   AGE
inventario-app-green-66b66dd7f4-jdrgx 1/1     Running            0           26m
inventario-app-green-66b66dd7f4-jskkl 1/1     Running            0           26m
inventario-app-green-77cfbc88f4-n5qpp 0/1     ContainerCreating  0           0s
inventario-app-green-77cfbc88f4-n5qpp 0/1     Running            0           3s
inventario-app-green-77cfbc88f4-n5qpp 1/1     Running            0           16s
inventario-app-green-66b66dd7f4-jskkl 1/1     Terminating      0           26m
inventario-app-green-77cfbc88f4-cswdp 0/1     Pending            0           0s
inventario-app-green-66b66dd7f4-jskkl 1/1     Terminating      0           26m
inventario-app-green-77cfbc88f4-cswdp 0/1     Pending            0           0s
inventario-app-green-77cfbc88f4-cswdp 0/1     ContainerCreating  0           0s
inventario-app-green-77cfbc88f4-cswdp 0/1     Running            0           3s
inventario-app-green-77cfbc88f4-cswdp 1/1     Running            0           16s
inventario-app-green-66b66dd7f4-jdrgx 1/1     Terminating      0           27m
inventario-app-green-66b66dd7f4-jdrgx 1/1     Terminating      0           27m
```

Evidencia: transición de 0/1 a 1/1 durante el rollout de Green.

Problema real	Corrección aplicada
BuildKit omitía inicialmente test	Producción copia el código desde la etapa test
ImagePullBackOff por SHA inexistente	Verificación de GHCR y uso de una etiqueta publicada
/health respondía 200 durante el arranque	Se implementó STARTUP_DELAY_SECONDS y una prueba 503/200
Trivy agotó el tiempo al descargar su base	Uso del repositorio público alternativo
PowerShell modificaba el JSON de kubectl patch	Uso de patch-green.json y patch-blue.json
Port-forward seguía conectado a Blue	Reinicio del port-forward después de cambiar el selector

Los errores se conservaron como parte de la evidencia. El intento fallido de despliegue también se incluye en el cálculo del change failure rate.

8. Métricas DORA propias

El lead time termina cuando Kubernetes registra NewReplicaSetAvailable. No se usa la hora de publicación de GHCR como sustituto. Los dos cambios medidos corresponden a imágenes SHA distintas.

SHA	Commit	Despliegue	Lead time
8a26dc3	25-jul 16:57:29	25-jul 17:01:00	00:03:31
85eaa0f	25-jul 17:02:01	25-jul 17:04:20	00:02:19

Métrica	Cálculo	Resultado	Nivel
Lead time	(00:03:31 + 00:02:19) / 2	00:02:55	Élite
Frecuencia	2 despliegues / 1 día	2 por día	Élite
CFR	1 corrección / 3 intentos x 100	33,33 %	Medio

La velocidad quedó en nivel Élite porque ambos cambios llegaron al clúster en pocos minutos y hubo dos despliegues correctos en un día. La estabilidad quedó en nivel Medio porque uno de los tres intentos necesitó corrección. La muestra es pequeña, por lo que estos niveles sirven para interpretar la práctica y deben recalcularse con más despliegues.

```
PS C:\Users\Fr4nk\Downloads\inventario-app\inventario-app> git show -s --format="%h | %cI | %s" 8a26dc39cb6ae2d23aa39387d80169f7fe649c2a 85eaa0fc3990f21c4fd8013e44ba51e0a30ca727
>> Write-Host
>> Import-Csv evidencias/dora-deployments.csv | Format-Table attempt_id,version,commit_at,deployed_at,lead_time,result -AutoSize
8a26dc3 | 2026-07-25T16:57:29-05:00 | Completar pipeline, readiness y documentacion final
85eaa0f | 2026-07-25T17:02:01-05:00 | Fijar imagen corregida en los despliegues

attempt_id version          commit_at          deployed_at          lead_time result
-----
1          initial-invalid-image 2026-07-24T17:30:02-05:00          failed-imagepullbackoff
2          corrected-release-1   2026-07-25T16:57:29-05:00 2026-07-25T17:01:00-05:00 00:03:31 success
3          corrected-release-2   2026-07-25T17:02:01-05:00 2026-07-25T17:04:20-05:00 00:02:19 success
```

Evidencia: commits, despliegues, lead times y resultado de los intentos.

Fuentes: evidencias/dora-deployments.csv, evidencias/despliegue-8a26dc3.txt y evidencias/despliegue-85eaa0f.txt. Los cambios se relacionan con los runs 30176640875 y 30176786798 de GitHub Actions.

9. Comandos para la demostración

Los siguientes bloques están preparados para PowerShell. Ejecutar desde la raíz del repositorio. Conviene usar dos terminales cuando se llegue al port-forward.

9.1 Herramientas y pruebas

```
Set-Location "C:\Users\Fr4nk\Downloads\inventario-app\inventario-app"

node --version
npm.cmd --version
git --version
docker --version
kubectl version --client
minikube version

npm.cmd ci
npm.cmd test
```

Debe observarse: las versiones instaladas y 6 pruebas correctas, 0 fallidas.

9.2 Construcción y contenedor Docker

```
docker build -t inventario-app:local .
docker images inventario-app

docker rm -f inventario-local 2>$null
docker run -d --name inventario-local -p 3000:3000 inventario-app:local
docker ps --filter name=inventario-local
docker logs inventario-local

curl.exe http://localhost:3000/health
curl.exe http://localhost:3000/version
curl.exe http://localhost:3000/api/products
```

Debe observarse: la etapa test durante el build, el contenedor Up y respuestas JSON con status ok, version v1, color blue y el catálogo.

9.3 Escaneo Trivy

```
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock `
-v trivy-cache:/root/.cache/ aquasec/trivy:0.72.0 image `
--quiet --timeout 10m `
--db-repository public.ecr.aws/aquasecurity/trivy-db:2 `
--scanners vuln --vuln-type os,library --format table `
--exit-code 1 --ignore-unfixed --severity CRITICAL `
inventario-app:local
```

Debe observarse: el resumen del análisis sin vulnerabilidades CRITICAL.

10. Demostración en Kubernetes

10.1 Iniciar o comprobar Minikube

```
minikube -p ci-cd status

# Ejecutar start solo si el perfil está detenido:
minikube -p ci-cd start --driver=docker --cpus=4 --memory=4096

kubectl get nodes
```

Debe observarse: el nodo ci-cd en estado Ready.

10.2 Crear el Secret y desplegar la versión base

```
kubectl create secret generic inventario-secret `
  --from-literal=API_KEY=CLAVE_LOCAL_DEMO `
  --dry-run=client -o yaml | kubectl apply -f -

kubectl get secret inventario-secret
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl rollout status deployment/inventario-app --timeout=240s
kubectl get deployment inventario-app -o wide
kubectl get pods -l app=inventario-app
```

Debe observarse: Secret con un dato, Deployment 2/2 y dos pods Running 1/1. No mostrar el valor de API_KEY.

10.3 Abrir el acceso local

Terminal 1:

```
kubectl port-forward service/inventario-service 8080:80
```

Terminal 2:

```
$URL = "http://127.0.0.1:8080"
curl.exe "$URL/health"
curl.exe "$URL/version"
curl.exe "$URL/api/products"
```

Debe observarse: health ok, la versión activa y la lista de productos. Mantener abierta la Terminal 1.

11. Blue-Green, readiness y métricas

11.1 Aplicar Blue y Green

```
kubectl apply -f k8s/blue-green/deployment-blue.yaml
kubectl apply -f k8s/blue-green/deployment-green.yaml
kubectl apply -f k8s/blue-green/service.yaml

kubectl rollout status deployment/inventario-app-blue --timeout=240s
kubectl rollout status deployment/inventario-app-green --timeout=240s
kubectl get pods -l app=inventario-app --show-labels
```

Debe observarse: dos pods Blue y dos pods Green en Running 1/1.

11.2 Promover Green y demostrar rollback

```
kubectl patch service inventario-service --type merge `
  --patch-file patch-green.json
kubectl get service inventario-service `
  -o jsonpath="{.spec.selector.slot}"

# Reiniciar el port-forward y luego consultar:
$URL = "http://127.0.0.1:8080"
curl.exe "$URL/version"

kubectl patch service inventario-service --type merge `
  --patch-file patch-blue.json
kubectl get service inventario-service `
  -o jsonpath="{.spec.selector.slot}"

# Reiniciar otra vez el port-forward:
curl.exe "$URL/version"
```

Debe observarse: Green devuelve v2/green. Después del rollback, Blue devuelve v1/blue. El port-forward se debe detener con Ctrl+C y abrir otra vez después de cada cambio.

11.3 Readiness y métricas

```
kubectl rollout restart deployment/inventario-app-green
kubectl get pods -l app=inventario-app,slot=green -w

# En otra terminal:
kubectl rollout status deployment/inventario-app-green --timeout=240s
kubectl get events --sort-by=.metadata.creationTimestamp |
  Select-String "inventario-app-green|Unhealthy" |
  Select-Object -Last 15

git log --pretty=format:"%h | %CI | %s" -10
Import-Csv evidencias/dora-deployments.csv |
  Format-Table attempt_id,version,commit_at,deployed_at,lead_time,result
```

Debe observarse: pods que pasan de 0/1 a 1/1, eventos 503 de readiness y la tabla DORA con dos despliegues correctos y un intento fallido.

11.4 Limpieza al terminar

```
docker rm -f inventario-local

# Detener port-forward con Ctrl+C antes de continuar.
kubectl delete -f k8s/blue-green/service.yaml --ignore-not-found
kubectl delete -f k8s/blue-green/deployment-blue.yaml --ignore-not-found
kubectl delete -f k8s/blue-green/deployment-green.yaml --ignore-not-found
minikube -p ci-cd stop
```

La limpieza es opcional durante la exposición. Si se necesita repetir la demostración, conviene dejar Minikube iniciado y ejecutar solo los comandos de comprobación.